

Scanned Code Report

AUDITAGENT

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Auditor Scan

#	Scan ID	✦	Date
	10		August 17, 2026
📦	Organization	📦	Repository
	CMTA		Rules
📦	Branch	📄	Commit Hash
	main		01632da0...951e204c

Contracts in scope

src/modules/AccessControlModuleStandalone.sol src/modules/MetaTxModuleStandalone.sol

src/modules/Ownable2StepERC165Module.sol src/modules/VersionModule.sol

src/registry/IdentityRegistryWhitelist.sol src/registry/abstract/IdentityRegistryWhitelistBase.sol

src/registry/abstract/IdentityRegistryWhitelistInvariantStorage.sol

src/registry/interfaces/IIdentityRegistryERC3643.sol src/rules/interfaces/AggregatorV3Interface.sol

src/rules/interfaces/IAddressList.sol src/rules/interfaces/IBalanceOf.sol src/rules/interfaces/IDecimals.sol

src/rules/interfaces/IERC2980.sol src/rules/interfaces/IERC7943NonFungibleCompliance.sol

src/rules/interfaces/IIdentityRegistry.sol src/rules/interfaces/ISanctionsList.sol

src/rules/interfaces/ITotalSupply.sol src/rules/interfaces/ITransferContext.sol

src/rules/interfaces/library/AddressListInterfaceId.sol src/rules/operation/RuleConditionalTransferLight.sol

src/rules/operation/RuleConditionalTransferLightMultiToken.sol

src/rules/operation/RuleConditionalTransferLightMultiTokenOwnable2Step.sol

src/rules/operation/RuleConditionalTransferLightOwnable2Step.sol

src/rules/operation/RuleMintAllowance.sol src/rules/operation/RuleMintAllowanceOwnable2Step.sol

src/rules/operation/abstract/RuleConditionalTransferLightApprovalBase.sol

src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

src/rules/operation/abstract/RuleConditionalTransferLightInvariantStorage.sol

src/rules/operation/abstract/RuleConditionalTransferLightMultiTokenBase.sol

src/rules/operation/abstract/RuleConditionalTransferLightMultiTokenInvariantStorage.sol

src/rules/operation/abstract/RuleMintAllowanceBase.sol

src/rules/operation/abstract/RuleMintAllowanceInvariantStorage.sol

src/rules/validation/abstract/RuleAddressSet/AddressSetBatchLib.sol

src/rules/validation/abstract/RuleAddressSet/RuleAddressSet.sol

src/rules/validation/abstract/RuleAddressSet/RuleAddressSetInternal.sol

src/rules/validation/abstract/RuleAddressSet/invariantStorage/RuleAddressSetInvariantStorage.sol

src/rules/validation/abstract/RuleAddressSet/invariantStorage/RuleAddressSetRolesStorage.sol

src/rules/validation/abstract/RuleAddressSet/invariantStorage/RuleBlacklistInvariantStorage.sol

src/rules/validation/abstract/RuleAddressSet/invariantStorage/RuleWhitelistInvariantStorage.sol

src/rules/validation/abstract/RuleERC2980/RuleERC2980Internal.sol

src/rules/validation/abstract/RuleERC2980/invariantStorage/RuleERC2980InvariantStorage.sol

src/rules/validation/abstract/base/RuleBlacklistBase.sol

src/rules/validation/abstract/base/RuleChainlinkPoRBase.sol

src/rules/validation/abstract/base/RuleERC2980Base.sol

src/rules/validation/abstract/base/RuleIdentityRegistryBase.sol

src/rules/validation/abstract/base/RuleMaxBalanceBase.sol

src/rules/validation/abstract/base/RuleMaxTotalSupplyBase.sol

src/rules/validation/abstract/base/RuleReceiverWhitelistBase.sol

src/rules/validation/abstract/base/RuleSanctionsListBase.sol

src/rules/validation/abstract/base/RuleSpenderWhitelistBase.sol

src/rules/validation/abstract/base/RuleWhitelistBase.sol

src/rules/validation/abstract/base/RuleWhitelistWrapperBase.sol

src/rules/validation/abstract/core/BalanceCapManager.sol

src/rules/validation/abstract/core/ChainlinkPoRFeedManager.sol

src/rules/validation/abstract/core/RuleNFTAdapter.sol

src/rules/validation/abstract/core/RuleTransferValidation.sol

src/rules/validation/abstract/core/RuleWhitelistShared.sol

src/rules/validation/abstract/core/TokenSupplyReader.sol

src/rules/validation/abstract/core/TotalSupplyCapManager.sol

src/rules/validation/abstract/invariant/RuleChainlinkPoRInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleIdentityRegistryInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleMaxBalanceInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleMaxTotalSupplyInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleReceiverWhitelistInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleSanctionsListInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleSharedInvariantStorage.sol

src/rules/validation/abstract/invariant/RuleSpenderWhitelistInvariantStorage.sol

src/rules/validation/deployment/RuleBlacklist.sol

src/rules/validation/deployment/RuleBlacklistOwnable2Step.sol

src/rules/validation/deployment/RuleChainlinkPoR.sol

src/rules/validation/deployment/RuleChainlinkPoROwnable2Step.sol

```
src/rules/validation/deployment/RuleERC2980.sol
src/rules/validation/deployment/RuleERC2980Ownable2Step.sol
src/rules/validation/deployment/RuleIdentityRegistry.sol
src/rules/validation/deployment/RuleIdentityRegistryOwnable2Step.sol
src/rules/validation/deployment/RuleMaxBalance.sol
src/rules/validation/deployment/RuleMaxBalanceOwnable2Step.sol
src/rules/validation/deployment/RuleMaxTotalSupply.sol
src/rules/validation/deployment/RuleMaxTotalSupplyOwnable2Step.sol
src/rules/validation/deployment/RuleReceiverWhitelist.sol
src/rules/validation/deployment/RuleReceiverWhitelistOwnable2Step.sol
src/rules/validation/deployment/RuleSanctionsList.sol
src/rules/validation/deployment/RuleSanctionsListOwnable2Step.sol
src/rules/validation/deployment/RuleSpenderWhitelist.sol
src/rules/validation/deployment/RuleSpenderWhitelistOwnable2Step.sol
src/rules/validation/deployment/RuleWhitelist.sol
src/rules/validation/deployment/RuleWhitelistOwnable2Step.sol
src/rules/validation/deployment/RuleWhitelistWrapper.sol
src/rules/validation/deployment/RuleWhitelistWrapperOwnable2Step.sol
```

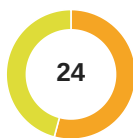
Code Statistics

 Findings
24

 Contracts Scanned
89

 Lines of Code
9764

Findings Summary



Total Findings

 High Risk (0)

 Medium Risk (13)

 Low Risk (11)

 Info (0)

 Best Practices (0)

Code Summary

This protocol provides a modular compliance and transfer-validation framework for tokenized real-world assets (RWAs) and security tokens, supporting standards such as ERC-3643, ERC-1404, ERC-2980 (Swiss Compliant Asset Token), and CMTAT. The architecture decouples rule enforcement from core token logic by providing standalone validation modules and rule engines that inspect transfers, mints, and burns for legal, regulatory, and institutional policy requirements.

Key architectural components and functionality include:

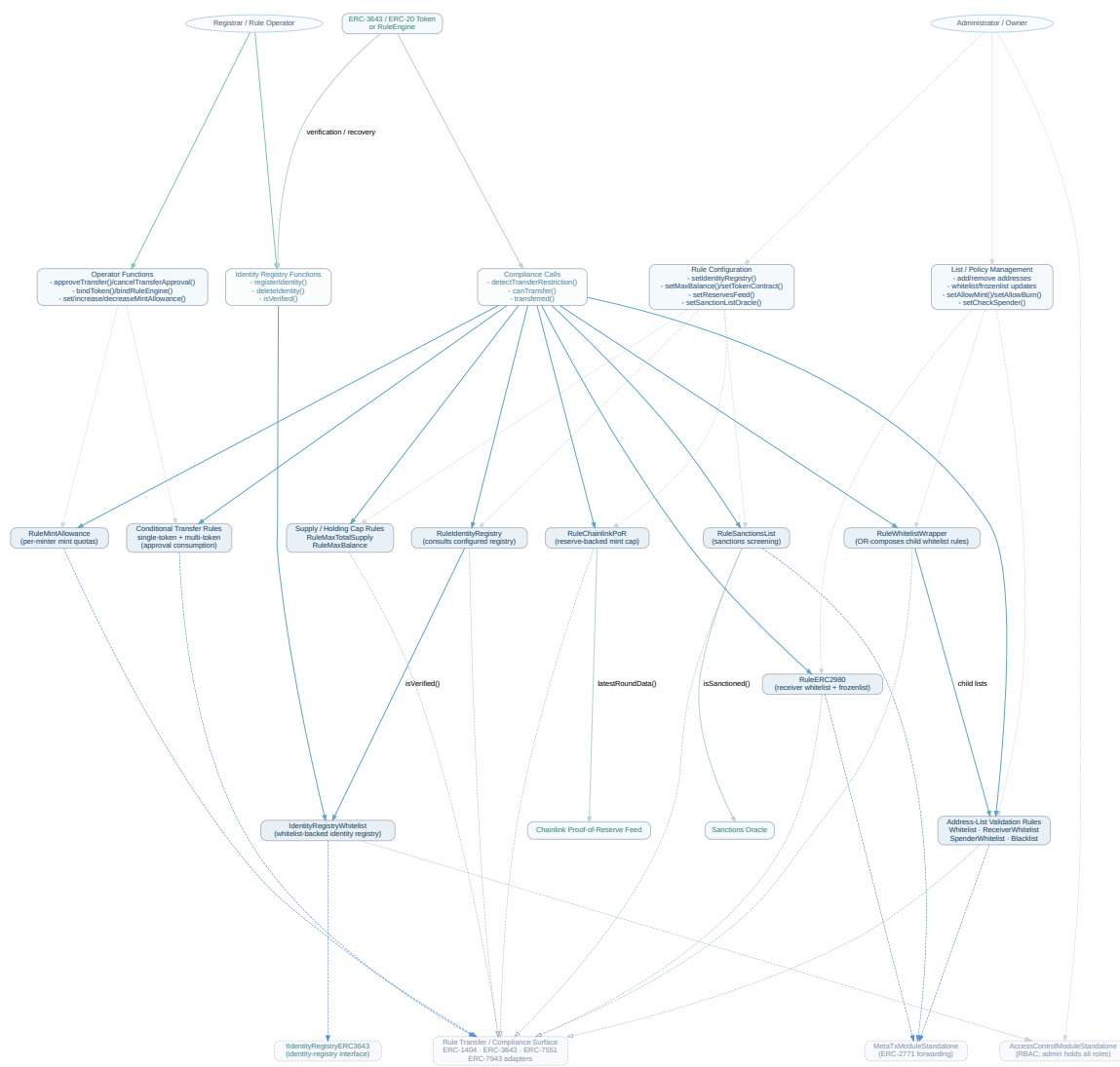
- Identity Registries: The IdentityRegistryWhitelist acts as a lightweight identity registry for ERC-3643 tokens, whitelisting verified investor wallets without storing personal identification data on-chain.
- Whitelist and Blacklist Rules: Contracts such as RuleWhitelist, RuleReceiverWhitelist, RuleSpenderWhitelist, and RuleBlacklist restrict transfer execution based on permissioned address sets managed via role-based access control. RuleWhitelistWrapper aggregates multiple child whitelist rules to perform composite compliance checks.
- ERC-2980 Swiss Compliance Rules: RuleERC2980 combines whitelist and frozenlist management to enforce Swiss asset token compliance, giving frozen list restrictions precedence over recipient whitelists.
- Balance and Supply Limits: RuleMaxBalance caps individual wallet token holdings with configurable exemption lists, RuleMaxTotalSupply enforces an absolute total supply cap, and RuleChainlinkPoR integrates Chainlink Proof of Reserve feeds to limit minting based on real-time off-chain reserve backing.
- Quota and Approval Controls: RuleMintAllowance tracks per-minter issuance limits to enforce minting quotas. RuleConditionalTransferLight and RuleConditionalTransferLightMultiToken require explicit off-chain or operator approvals for individual transfer tuple parameters prior to execution.
- Sanctions and Identity Verification: RuleSanctionsList interfaces with external oracles to block sanctioned addresses, while RuleIdentityRegistry verifies account status directly against ERC-3643 identity registries.

Main Entry Points

- registerIdentity: Called by an Identity Registrar to add a wallet address to the identity registry whitelist.
- deleteIdentity: Called by an Identity Registrar to remove a wallet address from the identity registry whitelist.
- addAddresses / addAddress: Called by Address List Adders to batch or individually append addresses to a whitelist, blacklist, or spender list.
- removeAddresses / removeAddress: Called by Address List Removers to batch or individually delete addresses from a whitelist, blacklist, or spender list.
- addWhitelistAddresses / addWhitelistAddress: Called by Whitelist Adders to whitelist addresses under ERC-2980 rules.
- removeWhitelistAddresses / removeWhitelistAddress: Called by Whitelist Removers to un-whitelist addresses under ERC-2980 rules.
- addFrozenlistAddresses / addFrozenlistAddress: Called by Frozenlist Adders to freeze addresses under ERC-2980 rules.
- removeFrozenlistAddresses / removeFrozenlistAddress: Called by Frozenlist Removers to unfreeze addresses under ERC-2980 rules.
- approveTransfer: Called by Transfer Approvers (Operators) to grant execution permission for a specific transfer tuple.
- cancelTransferApproval: Called by Transfer Approvers to revoke an outstanding transfer approval.
- resetApproval: Called by Transfer Approvers to clear all accumulated approvals for a transfer tuple.

- `approveAndTransferIfAllowed`: Called by Transfer Approvers to approve and execute an ERC-20 transfer in a single transaction.
- `transferred` / `created` / `destroyed`: Called by Bound Tokens or Rule Engines to record executed transfers, mints, or burns and update internal compliance states or allowances.
- `bindToken` / `bindRuleEngine` / `unbindRuleEngine`: Called by Compliance Managers to bind or unbind tokens and rule engines to compliance rules.
- `setMintAllowance` / `increaseMintAllowance` / `decreaseMintAllowance` / `clearMintAllowances`: Called by Allowance Operators to manage per-minter mint quotas.
- `addExemptAddress` / `removeExemptAddress` / `addExemptAddresses` / `removeExemptAddresses`: Called by Max Balance Managers to manage balance cap exemption lists.
- `setMaxBalance` / `setBalanceToken`: Called by Max Balance Managers to update wallet balance caps and target token contracts.
- `setSanctionListOracle` / `clearSanctionListOracle`: Called by Sanction List Managers to configure or disable external sanctions oracles.
- `setIdentityRegistry` / `clearIdentityRegistry` / `setCheckSender` / `setCheckSpender`: Called by Identity Registry Managers to configure target registries and toggle sender/spender verification flags.
- `setAllowMint` / `setAllowBurn` / `setCheckSpender`: Called by Mint/Burn or Spender Managers to enable or disable minting, burning, or spender validation across whitelist and ERC-2980 rules.
- `setReservesFeed` / `setTokenMetadata` / `setMaxStalenessSeconds`: Called by Chainlink PoR Managers to update Proof of Reserve feeds, token metadata, and maximum answer staleness limits.
- `setMaxTotalSupply` / `setTokenContract`: Called by Max Total Supply Managers to modify total supply ceilings and target token references.

Code Diagram



src/rules/operation/abstract/RuleMintAllowanceBase.sol

1 of 24 Findings

src/rules/operation/RuleMintAllowance.sol

src/rules/operation/RuleMintAllowanceOwnable2Step.sol

Mint quotas are not enforced when the bound integration reports mints through `created` or the three-argument `transferred` callback

• Medium Risk

`RuleMintAllowanceBase` performs allowance enforcement only in `_transferredFrom`, which is reached exclusively by `transferred(spender, from, to, value)`. In contrast, `created(address,uint256)` is empty and the three-argument `transferred(from,to,value)` invokes an empty `_transferred` helper; the corresponding three-argument views always return success. Therefore, if the caller bound to the rule reports a mint through either non-spender-aware callback, a token minter can complete mints without this rule checking or debiting `mintAllowance`. The intended CMTAT v3.3+ spender-aware integration is unaffected, but the contract does not fail closed when a bound integration uses the legacy callback shape.

Severity Note:

- The bound token's independent mint authorization permits amounts materially greater than the configured mint allowance.
- The minted token has meaningful value or its supply/backing ratio is a user-facing security invariant.
- The bound caller can use `created` or the 3-argument `transferred` callback despite the rule not advertising the full ERC-3643 compliance interface.

src/rules/operation/abstract/RuleMintAllowanceBase.sol

2 of 24 Findings

src/rules/operation/RuleMintAllowance.sol

src/rules/operation/RuleMintAllowanceOwnable2Step.sol

Mint quotas are shared across every token served by the bound RuleEngine

• Medium Risk

`RuleMintAllowanceBase` stores quota solely by minter address:

```
mapping(address minter => uint256 allowance) public mintAllowance;
```

The spender-aware callback deducts `mintAllowance[spender]`, but it receives no originating-token address. Binding is limited to one direct caller, yet the supported caller is normally a RuleEngine, and one RuleEngine can relay callbacks for multiple bound tokens. The single-caller guard therefore does not provide single-token isolation when that caller represents several assets.

If an allowance is assigned to a minter for token A, the same allowance is observed by the pre-flight and execution paths for token B behind the same engine. A minter holding mint permission on both tokens can mint token B even when no token-B-specific quota was granted, provided the amount is within the quota operationally intended for token A. The token-B mint also reduces the allowance available for token A, allowing one token's activity to deny issuance on another. Because the mapping and callback contain no token dimension, the rule cannot distinguish these cases, including where the governed tokens use different units or economic limits. The upstream RuleEngine explicitly supports multiple bound tokens and forwards stateful callbacks without an underlying-token argument. (raw.githubusercontent.com)

Severity Note:

- The bound RuleEngine indeed invokes this rule's four-argument callback for every served token and does not otherwise supply token-specific isolation; this is supported by the supplied validation evidence and RuleEngine description but the engine implementation is not included in the local contract excerpt.
- The shared allowance can be material relative to token B's issuance policy, and a token-B mint produces economically meaningful supply rather than a purely internal or valueless accounting unit.
- The token-level minter authorization on token B does not itself deliberately authorize use of token A's issuance allocation for token B.

✦ 3 of 24 Findings

src/rules/validation/abstract/base/RuleIdentityRegistryBase.sol

Early returns in `_detectTransferRestrictionFrom` silently bypass subclass transfer restrictions

• Medium Risk

`RuleIdentityRegistryBase` is an abstract base contract designed to be extended by specific rule implementations. Its `_detectTransferRestrictionFrom` function delegates to `_detectTransferRestriction` to apply baseline sender and receiver checks. However, if the `identityRegistry` is unset (`address(0)`) or the transfer is a burn (`to == address(0)`), the function returns `TRANSFER_OK` early, skipping the delegation entirely.

As explicitly documented in the sibling contract `RuleSanctionsListBase` (which previously suffered from and fixed this exact anti-pattern), nesting the delegation inside a conditional guard or returning early silently drops any security checks added by a subclass that overrides the `_detectTransferRestriction` hook. Consequently, if a derived rule adds custom logic (e.g., blacklists, locks, balance limits) to `_detectTransferRestriction` without redundantly overriding `_detectTransferRestrictionFrom`, attackers can completely bypass those custom security checks by executing the transfer via `transferFrom` (or `burnFrom`) when the registry is unset or during a burn.

Severity Note:

- A deployed subclass overrides `_detectTransferRestriction` with an independent restriction and does not also override `_detectTransferRestrictionFrom`.
- The subclass's restriction protects a meaningful user-facing transfer, lock, compliance, or asset-control guarantee rather than a purely cosmetic condition.
- The consuming token/rule-engine integration invokes the spender-aware `_detectTransferRestrictionFrom` path for the relevant delegated transfer or burn.

src/rules/operation/abstract/RuleConditionalTransferLightApprovalBase.sol

4 of 24 Findings

src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

src/rules/operation/abstract/RuleMintAllowanceBase.sol

Conditional-transfer approvals and mint quotas are not token-scoped across a shared RuleEngine or token rebinding

• Medium Risk

`RuleConditionalTransferLightApprovalBase` stores approvals under a hash of only `(from, to, value)`. `RuleConditionalTransferLightBase` permits a bound RuleEngine to consume those approvals, yet neither the approval key nor the execution callback identifies the token being relayed. Consequently, if the authorized engine relays more than one token, a matching transfer of token B can consume an approval recorded for token A. The same approval state also remains after `unbindToken` and is usable after a later `bindToken` unless the operator manually resets the relevant tuple.

`RuleMintAllowanceBase` similarly stores a single `mintAllowance[spender]` pool. Its spender-aware mint check and callback receive no token address, and binding an engine authenticates that engine rather than distinguishing the tokens it relays. Therefore, where one bound RuleEngine manages multiple tokens and a minter is authorized to mint them, quota assigned for token A can be used to pass the mint check for token B. Unbinding and rebinding also retain every minter's remaining quota unless the allowance operator manually clears that minter.

This breaks per-token authorization and issuance segregation. Use a separate rule/engine instance per protected token, or change the integration and storage keys to bind approvals and quotas to a token identity (and invalidate prior state atomically on migration).

Severity Note:

- The bound RuleEngine actually relays this rule's four-argument callbacks for more than one token, despite the explicit single-token-engine deployment warning.
- For the material mint scenario, a minter with remaining allowance intended for TokenA is also authorized to invoke minting for TokenB.
- TokenB does not independently impose an effective per-minter cap that prevents the cross-token quota from producing excess issuance.

5 of 24
Findings

src/rules/validation/abstract/base/RuleChainlinkPoRBase.sol
src/rules/validation/abstract/core/TokenSupplyReader.sol
src/rules/validation/abstract/base/RuleMaxTotalSupplyBase.sol
src/rules/validation/abstract/core/TotalSupplyCapManager.sol

Static token binding in shared RuleEngine deployments checks the wrong token's supply

• Medium Risk

Both `RuleChainlinkPoRBase` and `RuleMaxTotalSupplyBase` read `totalSupply()` from a statically configured `tokenContract`, while their standard compliance entrypoints do not verify that the token invoking the rule matches that address.

In `RuleChainlinkPoRBase`, the rule's external validation endpoints (`canTransfer`, `transferred`, and `detectTransferRestriction`) do not verify that the token invoking the rule (`msg.sender`) matches the configured `tokenContract`. If `Token B` binds to a rule configured for `Token A`, any mint requests on `Token B` evaluate `Token A`'s `totalSupply()`. As `Token B` mints, `Token A`'s supply remains static and unaffected. Consequently, `Token B` can mint an infinite number of unbacked tokens without ever exceeding the PoR limit, entirely bypassing the underlying reserve invariant.

`RuleMaxTotalSupplyBase` has the same caller-context gap when attached to a shared `RuleEngine`. The standard compliance interfaces (`canTransfer`, `detectTransferRestriction`) do not pass the address of the token being transferred to the rule, so a rule configured with `tokenContract` pointing to a specific primary token will evaluate mints for all integrated tokens against that one address. When an auxiliary token mints, its new issuance does not increase the hardcoded token's `totalSupply()`. The rule's supply check therefore never reflects the auxiliary token's issuances, and users can mint an infinite number of the auxiliary token across multiple transactions as long as each individual mint value does not exceed the remaining headroom of the hardcoded token.

Severity Note:

- For direct monetary loss rather than solely an issuance-integrity violation, the excess Token-B tokens must be economically valuable—e.g., transferable, saleable, accepted as collateral, or redeemable against protocol/user assets.
- The Token-B minter can mint to itself, an accomplice, or otherwise cause the unauthorized excess issuance to enter circulation. The provided code establishes the cap bypass but does not show Token B's mint authorization or downstream redemption/market integrations.

✦ 6 of 24 Findings

src/rules/validation/abstract/core/RuleNFTAdapter.sol

RuleNFTAdapter inconsistently handles owner-initiated transfers across context and ERC-7943 spender-aware hooks

• Medium Risk

`RuleNFTAdapter.transferred(FungibleTransferContext)` and `transferred(MultiTokenTransferContext)` normalize `ctx.sender == ctx.from` to the direct `_transferred(from, to, value)` hook. In contrast, the ERC-7943 five-argument `transferred(spender, from, to, tokenId, value)` and `detectTransferRestrictionFrom(spender, from, to, tokenId, value)` unconditionally use their spender-aware internal hooks. This produces contradictory outcomes for a standard owner-initiated `transferFrom` where `spender == from`. In particular, `RuleSpenderWhitelistBase` intentionally makes direct transfers a no-op but rejects delegated transfers when the spender is unlisted; therefore the context path accepts an unlisted owner while the ERC-7943 spender-aware path rejects that same owner transfer. Normalize self-spender handling consistently across these adapter entrypoints, or consistently retain spender semantics in the context path, so supported integrations cannot obtain conflicting compliance outcomes for the same transfer.

✦ 7 of 24 Findings

src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

src/rules/operation/RuleConditionalTransferLight.sol

src/rules/operation/RuleConditionalTransferLightOwnable2Step.sol

Single-token approvals are reusable across tokens behind a shared RuleEngine

• Medium Risk

The rule's bound token does not form part of an approval's storage key. `approveTransfer` records an approval under `_transferHash(from, to, value)`, while the restriction and execution paths read and consume that same token-agnostic key:

```
bytes32 transferHash = _transferHash(from, to, value);
uint256 count = approvalCounts[transferHash];
```

At the same time, `bindRuleEngine` independently authorizes a RuleEngine to execute either `transferred` overload. Those callbacks contain no originating-token parameter and simply invoke `_transferred(from, to, value)`. Consequently, binding a RuleEngine that serves token A and token B makes transfers of both tokens operate on the same approval bucket.

For example, after an operator approves `(Alice, Bob, 100)` while the rule is intended to govern token A, Alice can execute the same tuple using token B. The shared engine's read path observes the token-A approval and its write callback consumes it because the rule cannot distinguish which token caused the callback. Token B therefore completes a transfer without a token-B-specific conditional approval, while the approval intended for token A is consumed and the later token-A transfer is denied. A RuleEngine can support multiple token bindings, and its rule callback does not carry the originating token address. ([raw.githubusercontent.com](https://raw.githubusercontent.com/NethermindSec/auditagent/master/src/rules/operation/RuleConditionalTransferLightOwnable2Step.sol))

src/rules/operation/RuleMintAllowance.sol

8 of 24 Findings

src/rules/operation/RuleMintAllowanceOwnable2Step.sol

src/rules/operation/abstract/RuleMintAllowanceBase.sol

Mint allowances are shared across all tokens serviced by a bound multi-token RuleEngine

• Medium Risk

`RuleMintAllowanceBase` binds only the immediate callback caller and stores quotas solely by minter address:

```
mapping(address minter => uint256 allowance) public mintAllowance;
```

During a mint, `_transferredFrom` deducts from `mintAllowance[spender]` without incorporating the originating token address. In the intended RuleEngine topology, the bound caller is the RuleEngine rather than the token. A RuleEngine can be bound to multiple tokens and forwards `transferred(spender, from, to, value)` to child rules without forwarding the originating token address, so every token behind that engine reaches this rule through the same caller and the same allowance mapping. (raw.githubusercontent.com)

Consequently, a quota operationally assigned to a minter for token A can be consumed by minting token B. If the minter is authorized to mint both tokens, a nonzero allowance intended for token A also authorizes issuance of token B even when token B's intended allowance is zero. Conversely, minting token B can consume the quota needed for token A and prevent subsequent authorized issuance. The problem is especially significant when the tokens have different decimals, valuations, issuers, or reserve requirements because the shared raw allowance units have no token-specific meaning. Binding only one RuleEngine address does not provide single-token isolation when that RuleEngine itself services multiple tokens.

Severity Note:

- The shared allowance is operationally intended to be token-specific, rather than deliberately representing one aggregate issuance budget across every token serviced by the RuleEngine.
- Token B's issuance has material economic, regulatory, reserve, or supply-control consequences; the supplied code establishes the cap bypass but does not establish a particular token's valuation or redemption mechanics.
- The applicable token mint flow uses the documented four-argument spender-aware callback, as required by this rule's design.

9 of 24
Findings

```
src/rules/validation/abstract/base/RuleBlacklistBase.sol
src/rules/validation/abstract/base/RuleERC2980Base.sol
src/rules/validation/abstract/base/RuleSanctionsListBase.sol
src/rules/validation/abstract/base/RuleSpenderWhitelistBase.sol
src/rules/validation/abstract/base/RuleWhitelistBase.sol
src/rules/validation/abstract/base/RuleWhitelistWrapperBase.sol
src/rules/validation/abstract/core/RuleWhitelistShared.sol
```

Three-argument ERC-3643 hooks bypass spender-based compliance controls

• Medium Risk

Spender-specific restrictions are enforced exclusively through the four-argument transfer path. Any integration that invokes only the three-argument `IERC3643ComplianceContract` path cannot communicate the actual spender to these rules and therefore cannot enforce the configured spender policy.

The most direct instance is `RuleSpenderWhitelistBase`, whose three-argument callback and corresponding detector unconditionally accept the operation:

```
function transferred(address, address, uint256)
    public
    view
    override(IERC3643ComplianceContract)
{}

function _detectTransferRestriction(address, address, uint256)
    internal
    pure
    virtual
    override
    returns (uint8)
{
    return uint8(IERC1404Extend.REJECTED_CODE_BASE.TRANSFER_OK);
}
```

Its whitelist check is reachable only through `_detectTransferRestrictionFrom(spender, from, to, value)`. The same context loss affects the spender branches of the other affected rules: code 23 for a non-whitelisted spender, code 38 for a blacklisted spender, code 32 for a sanctioned spender, and code 62 for a frozen spender can only be produced by their four-argument paths.

Consequently, when a token or compliance engine uses only the three-argument path for both `transfer` and `transferFrom`, a restricted spender with an existing allowance can call `transferFrom` between otherwise eligible endpoints. `RuleSpenderWhitelist` becomes entirely ineffective, while the other affected rules continue checking `from` and `to` but silently omit the spender restriction. This permits delegated movements by operators that the configured compliance policy explicitly classifies as unapproved, blacklisted, sanctioned, or frozen.

Severity Note:

- A deployed token or compliance engine invokes only `transferred(from,to,value)` or its corresponding three-

argument restriction query for transferFrom operations.

- No separate token-level or adapter-level spender validation is performed before the transfer is finalized.
- The deployment enables a spender-sensitive policy and at least one restricted spender retains a usable allowance from a compliant holder.

10 of 24
Findings

src/rules/validation/abstract/core/ChainlinkPoRFeedManager.sol
src/rules/validation/abstract/base/RuleChainlinkPoRBase.sol

Future-dated PoR timestamps bypass the configured freshness limit

• Medium Risk

`ChainlinkPoRFeedManager._maxBackedSupply` marks a feed stale only when `block.timestamp > updatedAt` and the elapsed time exceeds `maxStalenessSeconds`. It does not reject `updatedAt > block.timestamp`. A faulty or compromised feed can therefore return an old but high reserve answer with a sufficiently future nonzero timestamp; the rule treats it as fresh and can authorize mints from that stale reserve value until the future timestamp and subsequent staleness window elapse. The PoR check should fail closed when `updatedAt` is in the future.

Severity Note:

- The configured rule is actually enforced on every relevant token mint path, as described in the finding preconditions.
- The token's reserve backing is economically meaningful to holders or redeemers, such that supply minted above actual reserves creates material dilution or insolvency risk.
- The faulty/compromised PoR response contains an overstated reserve answer and a future timestamp; a future timestamp alone does not increase mint headroom.

11 of 24
Findings

src/rules/validation/abstract/base/RuleMaxBalanceBase.sol
src/rules/validation/abstract/core/BalanceCapManager.sol
src/rules/validation/abstract/base/RuleMaxTotalSupplyBase.sol
src/rules/validation/abstract/base/RuleChainlinkPoRBase.sol

Balance/supply caps double-count the transferred amount when the token invokes the write hook after updating balances (ERC-3643 post-transfer `transferred`)

• Medium Risk

The cap rules read the observed token's live state inside the write hook and compare it against the value being moved:

```
// BalanceCapManager._capExceeded
(balanceAvailable, balance) = _balanceOf(to);
return (true, balance > cap || value > cap - balance);

// TotalSupplyCapManager._capExceeded
(supplyAvailable, currentSupply) = _currentSupply();
return (true, currentSupply > cap || value > cap - currentSupply);
```

This is only correct if the token calls `transferred(...)` BEFORE mutating balances/supply, which the code comments acknowledge as an assumption ("Assumes the token calls this BEFORE moving the value ... a token notifying afterwards would halve the effective cap").

However the project explicitly supports ERC-3643 tokens through a `RuleEngine` wrapper, and ERC-3643/T-REX tokens invoke the compliance hook AFTER the state change: `canTransfer(...)` is consulted first, then `_transfer(...)` executes, then `compliance.transferred(from, to, amount)` is called (and `created(to, amount)` after `_mint`). Under that ordering:

- `balanceOf(to)` already includes `value`, so the effective per-holder cap becomes `balanceOf_before(to) + 2 * value <= maxBalance`.
- `totalSupply()` already includes the minted amount, so the effective supply cap becomes `supply_before + 2 * value <= maxTotalSupply`, and the same double counting applies to `RuleChainlinkPoRBase`'s reserve headroom check `(currentSupply > backedSupply || value > backedSupply - currentSupply)`.

The read path (`detectTransferRestriction`, `canTransfer`, `remainingCapacity`, `maxBackedSupply`) is evaluated before the state change and therefore reports the transfer/mint as compliant, while the write hook then reverts on exactly the same parameters. The result is an inconsistency between the pre-flight answer and the enforcement answer, and the silent rejection of transfers and mints that are actually within the configured cap (for example:

`maxBalance = 100`, receiver balance `0`, transfer of `60` is approved by `canTransfer` but reverts in `transferred` because the post-update balance `60` plus `value 60` exceeds `100`). Whole legitimate issuance/transfer flows can be blocked, and the last chunk of headroom under any cap becomes unreachable.

12 of 24
Findings

src/rules/operation/RuleConditionalTransferLight.sol

src/rules/operation/RuleConditionalTransferLightOwnable2Step.sol

src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

src/rules/operation/abstract/RuleConditionalTransferLightApprovalBase.sol

Single-token approvals can authorize transfers of another token through a shared RuleEngine

• Medium Risk

Single-token conditional approvals are keyed only by `(from, to, value)`:

```
bytes32 transferHash = _transferHash(from, to, value);
```

`RuleConditionalTransferLightBase` separately authorizes the configured `ruleEngine` to execute and consume these approvals. If that engine services multiple tokens, every token is evaluated against the same `approvalCounts` mapping because the RuleEngine callback does not identify which token originated the operation. The upstream engine supports multiple bound tokens and forwards only the transfer participants and amount to each rule. ([raw.githubusercontent.com](https://raw.githubusercontent.com/ethereum-optimism/optimism/master/packages/contracts-bedrock/contracts/RuleConditionalTransferLightBase.sol))

For example, an operator can approve `(Alice, Bob, 100)` for the rule's bound token A. If token B uses the same RuleEngine, a transfer of 100 token B from Alice to Bob returns `TRANSFER_OK`, and the post-transfer callback from the engine consumes the approval intended for token A. This both bypasses token B's conditional-transfer policy and removes token A's valid approval. The bound-token field does not prevent this path because execution is authorized solely by the shared RuleEngine address.

13 of 24
Findings

src/rules/validation/abstract/base/RuleMaxTotalSupplyBase.sol

src/rules/validation/abstract/base/RuleChainlinkPoRBase.sol

src/rules/validation/abstract/base/RuleMaxBalanceBase.sol

src/rules/validation/abstract/core/TotalSupplyCapManager.sol

src/rules/validation/abstract/core/ChainlinkPoRFeedManager.sol

src/rules/validation/abstract/core/BalanceCapManager.sol

Supply- and balance-capped rules never bind to the calling token, so a shared instance or a repointed token silently voids the cap

• Medium Risk

`RuleMaxTotalSupplyBase`, `RuleChainlinkPoRBase` and `RuleMaxBalanceBase` evaluate their caps against an address stored in configuration (`TotalSupplyCapManager.tokenContract`, `ChainlinkPoRFeedManager.tokenContract`, `BalanceCapManager.balanceToken`) while the enforcement hooks accept the transfer parameters from whoever calls them:

```
function transferred(address from, address to, uint256 value) public view
override(IERC3643IComplianceContract) {
    _transferred(from, to, value);
}
```

No hook verifies that `msg.sender` is the configured token, and nothing ties the cap to the asset actually being moved. Two operational consequences follow:

1. A single rule instance placed behind a `RuleEngine` that serves several tokens, or reused across several engines, enforces the ceiling only against the configured token. Mints of the other tokens are checked against a supply, reserve feed and balance belonging to an unrelated asset, so `RuleMaxTotalSupply`'s ceiling, `RuleChainlinkPoR`'s reserve backing and `RuleMaxBalance`'s per-holder cap can all be satisfied trivially (for instance while the configured token's supply is 0 and the receiver's balance in the configured token is 0), and the intended restriction is not applied to the token that is actually minting.
2. `setTokenContract`, `setTokenMetadata` and `setBalanceToken` let the manager repoint the observation target at any callable contract at any time. Pointing a supply cap at a token with a small `totalSupply()`, or a balance cap at a token in which the receiver holds nothing, neutralises the restriction for the protected token while leaving the rule bound and apparently active; only a `TokenContractUpdated` / `TokenMetadataUpdated` / `MaxBalanceTokenUpdated` event distinguishes the two situations.

Because the caps are the only supply- and concentration-limiting mechanism these rules provide, both cases result in unbacked or over-cap issuance being reported as `TRANSFER_OK` by the read path and accepted by the write hook.

Severity Note:

- A token or shared `RuleEngine` uses one of these rules as an authoritative pre-transfer/pre-mint enforcement hook.
- For the PoR and total-supply variants, an authorized mint path exists and a mint above the configured limit has economic, compliance, or backing consequences.

- The deployment intends the configured cap/backing guarantee to apply to the calling token rather than treating the configured observed token as an intentionally independent reference asset.

src/rules/validation/abstract/base/RuleIdentityRegistryBase.sol

14 of 24 Findings

src/rules/validation/deployment/RuleIdentityRegistry.sol

src/rules/validation/deployment/RuleIdentityRegistryOwnable2Step.sol

Uncontained identity-registry failures revert non-burn validation and block transfers or mints

• Low Risk

`RuleIdentityRegistryBase.setIdentityRegistry` accepts any nonzero address without verifying that it implements a callable `isVerified(address)` function. Its restriction paths then make unguarded typed calls to `registry.isVerified`. If the configured address has no code, returns malformed/empty data, or a previously valid registry begins reverting, the call reverts instead of producing a restriction code. All non-burn direct and spender-aware checks consequently fail, and the corresponding `transferred` hooks revert, blocking normal transfers and mints in an integration that enforces the rule. Burns remain exempt because both restriction paths return successfully when `to == address(0)` before consulting the registry.

15 of 24 Findings  src/rules/operation/abstract/RuleConditionalTransferLightApprovalBase.sol
src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

Single-token conditional-transfer approvals are not token-scoped, so a bound RuleEngine serving more than one token (or a token rebinding) lets an approval be consumed by an unrelated transfer

• Low Risk

`RuleConditionalTransferLightApprovalBase` keys every approval on `_transferHash(from, to, value)`:

```
assembly ("memory-safe") {
    let ptr := mload(0x40)
    mstore(ptr, shl(96, from))
    mstore(add(ptr, 0x20), shl(96, to))
    mstore(add(ptr, 0x40), value)
    hash := keccak256(ptr, 0x60)
}
```

There is no token dimension in the key. At the same time `RuleConditionalTransferLightBase` authorizes two independent callers for the consumption hooks:

```
function isTransferExecutor(address caller) public view virtual returns (bool) {
    return isTokenBound(caller) || (caller != address(0) && caller == ruleEngine);
}
```

`bindRuleEngine(address)` performs no validation that the engine relays exactly one token; it only checks non-zero and not-already-bound. A `RuleEngine` is multi-tenant by construction, and every token it serves calls the same `transferred(...)` hook on this rule. Consequently, an approval recorded by an operator for a `(from, to, value)` tuple intended for token A is fully consumable by a transfer of token B with the same sender, receiver, and amount: the hash lookup cannot distinguish them and `_transferred` decrements the single shared bucket.

The same collision arises across the token lifecycle: `unbindToken()` does not clear `approvalCounts` and does not clear `ruleEngine`, so approvals recorded while token A was bound remain in storage and become consumable once token B is bound, and the previously authorized engine remains an authorized executor until `unbindRuleEngine()` is called.

The constraint that only a single-token engine may be bound is enforced solely by NatSpec warnings; nothing in the code prevents an administrator from wiring a multi-tenant engine, after which the rule authorizes transfers it never approved for that token.

✦ 16 of 24 Findings

src/rules/validation/abstract/base/RuleSpenderWhitelistBase.sol

RuleSpenderWhitelist does not enforce spender policy through spender-less compliance hooks

• Low Risk

RuleSpenderWhitelistBase deliberately returns TRANSFER_OK from `_detectTransferRestriction(address,address,uint256)` and makes `transferred(address,address,uint256)` a no-op. Its only membership test is in `_detectTransferRestrictionFrom(spender,from,to,value)`. Consequently, if a token or compliance integration routes `transferFrom` through only the spender-less three-argument path, an unlisted spender with an existing allowance can transfer between otherwise permitted endpoints because the rule is never given, and never checks, the actual spender. Three-argument pre-flight calls also necessarily report only this spender-less result and must not be treated as authorization for delegated transfers.

Severity Note:

- A deployed token or compliance adapter can route delegated `transferFrom` operations solely through the three-argument spender-less hook while this rule is configured as the spender-authorization control.
- At least one holder has granted an allowance to an unlisted spender, and the whitelist is intended to prohibit that spender from using that allowance despite the holder's ERC-20 approval.

17 of 24
Findings

src/rules/operation/abstract/RuleConditionalTransferLightBase.sol

src/rules/operation/abstract/RuleConditionalTransferLightMultiTokenBase.sol

approveAndTransferIfAllowed records an approval before executing the transfer, leaving a permanently spendable approval if the token does not call back the consumption hook

• Low Risk

Both helper implementations intentionally invert the checks-effects-interactions order so that the approval exists while the token performs its compliance callback:

```
function approveAndTransferIfAllowed(address from, address to, uint256 value) public virtual
onlyTransferApprover returns (bool) {
    address token = getTokenBound();
    require(token != address(0), RuleConditionalTransferLight_TokenNotBound());
    approveTransfer(from, to, value);
    uint256 allowed = IERC20(token).allowance(from, address(this));
    require(allowed >= value, RuleConditionalTransferLight_InsufficientAllowance(token,
    from, allowed, value));
    IERC20(token).safeTransferFrom(from, to, value);
    return true;
}
```

The correctness of this pattern depends entirely on the bound token actually invoking `transferred(...)` on this rule during `transferFrom`, which is the only place the freshly created approval is consumed. There is no post-condition verifying that the approval count returned to its pre-call value. Whenever the callback does not happen — the bound token is a plain ERC-20, the token's rule-engine slot points somewhere else, the rule was bound with `bindToken` for the helper but the engine was never bound / was later unbound with `unbindRuleEngine`, or the token's validation module is temporarily deactivated — the transfer still succeeds and `approvalCounts[_transferHash(from, to, value)]` is left incremented.

That residual approval is indistinguishable from an operator-created one and authorizes a later, never-approved transfer of exactly `(from, to, value)`: the holder `from` (or any third party holding an ERC-20 allowance from `from`) can execute an additional transfer of the same amount to the same recipient and it will pass `detectTransferRestriction` and `_transferred`. This breaks the property that a normal transfer cannot execute without a matching operator-issued approval. The only remediation is an explicit `resetApproval` / `cancelTransferApproval` call, which requires the operator to notice the leftover count.

✦ 18 of 24 Findings

src/rules/validation/abstract/base/RuleWhitelistWrapperBase.sol

RuleWhitelistWrapper can be bricked by adding a child rule that is not an IAddressList

• Low Risk

`RuleWhitelistWrapperBase` treats every configured child rule as an `IAddressList` during transfer validation, but it never verifies that assumption before making the external call.

```
function _detectTransferRestrictionForTargets(address[] memory targetAddress)
    internal
    view
    virtual
    returns (bool[] memory)
{
    uint256 rulesLength = rulesCount();
    ...
    for (uint256 i = 0; i < rulesLength; ++i) {
        bool[] memory isListed = IAddressList(rule(i)).areAddressesListed(targetAddress);
        ...
    }
}
```

The inherited rule-management path only prevents zero addresses and duplicates, so a rules manager can add a real `IRule` that is not a whitelist-style address list, such as `RuleMaxTotalSupply`. Once such a child is inserted before a later whitelist child, any transfer check whose `from`/`to`/`spender` set is not fully resolved by earlier children will revert on the blind `areAddressesListed` call instead of continuing the wrapper's OR-resolution across the remaining child whitelists.

This is an external-integration boundary failure between the wrapper and its child rules. The PoC shows a transfer that is valid when the wrapper contains two genuine `RuleWhitelist` children, then becomes impossible after inserting a real `RuleMaxTotalSupply` child between them. The failure is not limited to read helpers: the enforcement path `transferred(...)` reverts too, so an otherwise permitted transfer can no longer pass through the wrapper.

✦ 19 of 24 Findings

src/rules/validation/abstract/base/RuleWhitelistWrapperBase.sol

RuleWhitelistWrapperBase does not implement IAddressList, breaking wrapper nesting and composability

• Low Risk

The `RuleWhitelistWrapperBase` contract is designed to aggregate multiple child whitelist rules, evaluating participant arrays and treating their returned membership vectors as authoritative under OR composition. While it acts logically as a composite whitelist and properly implements the `IIdentityRegistryVerified` interface, it fails to implement the `IAddressList` interface itself (specifically omitting the `areAddressesListed` and `isAddressListed` external views).

Because of this omission, a wrapper cannot be used as a child rule of another wrapper. If an admin attempts to nest wrappers (e.g., to create a hierarchical OR-composition combining a 'US Investors Wrapper' and an 'EU Investors Wrapper'), any token transfer evaluated by the top-level parent wrapper will trigger a `STATICCALL` to the child wrapper's missing `areAddressesListed` selector. Since the child wrapper has no fallback function, the call will revert, causing a complete denial of service for all transfers relying on the parent wrapper.

The wrapper already contains the exact internal logic required to satisfy this interface (`_detectTransferRestrictionForTargets`), but its omission from the public ABI breaks the intended rule composability and creates a configuration hazard that silently bricks transfers.

src/rules/validation/abstract/base/RuleWhitelistWrapperBase.sol

✦ 20 of 24 Findings

src/rules/validation/abstract/base/RuleBlacklistBase.sol

src/rules/validation/deployment/RuleWhitelistWrapper.sol

RuleWhitelistWrapper treats raw membership from an incompatible RuleBlacklist child as whitelist eligibility

• Low Risk

`RuleWhitelistWrapperBase` ORs raw `areAddressesListed` answers from every configured child and interprets a true result as endpoint eligibility. A `RuleBlacklist` is a valid `IRule` for `addRule` and, through `RuleAddressSet`, exposes `areAddressesListed`; however, its listed addresses are meant to be denied. If a blacklist child contains A and another wrapper child lists B, the wrapper resolves both A and B as eligible and permits A to transfer to B. It also reports `isVerified(A) == true`. The wrapper does not constrain children to compatible endpoint allow-list semantics or otherwise encode list polarity.

Severity Note:

- Material financial loss beyond a compliance-policy breach would depend on the blacklisted account holding transferable token balances and being able to find a counterparty resolved by another child allow-list.
- The code supplied does not prove that all uses of this wrapper enforce a blacklist for stolen assets or otherwise make each bypass equivalent to direct third-party fund theft.

src/rules/operation/RuleMintAllowance.sol

21 of 24 Findings

src/rules/operation/RuleMintAllowanceOwnable2Step.sol

src/rules/operation/abstract/RuleMintAllowanceBase.sol

RuleMintAllowance exposes fail-open 3-argument mint preflight views that disagree with enforcement

• Low Risk

`RuleMintAllowanceBase` hardcodes the standard 3-argument ERC-1404 / ERC-3643 eligibility views to report success even though mint quota enforcement happens only on the spender-aware 4-argument path.

```
function detectTransferRestriction(address, address, uint256)
    public
    view
    virtual
    override(IERC1404)
    returns (uint8)
{
    return uint8(IERC1404Extend.REJECTED_CODE_BASE.TRANSFER_OK);
}

function canTransfer(address, address, uint256)
    public
    view
    virtual
    override(IERC3643ComplianceRead)
    returns (bool)
{
    return true;
}

function transferred(address spender, address from, address to, uint256 value)
    public
    virtual
    override(IRuleEngine)
    onlyBoundToken
{
    _transferredFrom(spender, from, to, value);
}
```

When this rule is installed behind `RuleEngine`, the engine and the token both forward the 3-argument preflight unchanged. The result is that `token.detectTransferRestriction(address(0), to, amount)` and `token.canTransfer(address(0), to, amount)` report a mint as allowed even when `mintAllowance[spender]` is zero and the actual `mint()` call reverts through the 4-argument enforcement path. The PoC shows this on the real CMTAT + RuleEngine integration: token-level preflight returns success for a zero-quota minter, while the mint itself still reverts. Integrations that rely on the standard token-level preflight API therefore receive a false green light and submit transactions that cannot succeed.

✦ 22 of 24 Findings

src/rules/validation/abstract/base/RuleSanctionsListBase.sol

A misbehaving sanctions oracle blocks all token activity and makes the read path revert

• Low Risk

`RuleSanctionsListBase` calls the externally supplied oracle without any failure handling:\n\n`solidity\nif (from != address(0) && oracle.isSanctioned(from)) {\n return CODE_ADDRESS_FROM_I` is an arbitrary third-party contract (the Chainalysis oracle in the intended deployment). If it reverts, is upgraded to a contract that reverts, returns non-boolean data of unexpected length, or consumes more gas than the caller forwards, the `STATICCALL` failure propagates: `_detectTransferRestriction` and `_detectTransferRestrictionFrom` revert instead of returning a restriction code. Because these are also reached from the ERC-1404 read path, `detectTransferRestriction` and `canTransfer` — which the project documentation requires to never revert — will revert, and `transferred` reverts, halting every transfer, mint and burn of the bound token for as long as the oracle is unhealthy. There is no cached fallback value, no bounded gas stipend and no explicit fail-open/fail-closed policy selector for this scenario; recovery requires a privileged transaction to repoint or clear the oracle.

✦ 23 of 24 Findings

src/rules/validation/abstract/core/BalanceCapManager.sol

src/rules/validation/abstract/core/TokenSupplyReader.sol

src/rules/validation/abstract/core/ChainlinkPoRFeedManager.sol

src/rules/validation/abstract/base/RuleMaxBalanceBase.sol

src/rules/validation/abstract/base/RuleMaxTotalSupplyBase.sol

src/rules/validation/abstract/base/RuleChainlinkPoRBase.sol

Short successful dependency return data can escape try/catch and revert restriction checks

• Low Risk

`BalanceCapManager._balanceOf`, `TokenSupplyReader._currentSupply`, and the typed feed calls in `ChainlinkPoRFeedManager._maxBackedSupply` rely on high-level typed calls in `try/catch`. If a configured code-bearing dependency later successfully returns fewer bytes than its declared return type requires—for example after a proxy implementation change—ABI decoding fails in the caller outside the `catch` handler. Consequently, max-balance checks, supply-cap mint checks, or PoR feed checks can revert instead of returning their unavailable-data restriction codes. The rule remains conservative because enforcement reverts, but its documented revert-free read-path behavior and code-based RuleEngine/pre-flight integration are lost.

24 of 24
Findings

src/rules/validation/abstract/core/TokenSupplyReader.sol

src/rules/validation/abstract/core/BalanceCapManager.sol

src/rules/validation/abstract/base/RuleMaxBalanceBase.sol

src/rules/validation/abstract/base/RuleMaxTotalSupplyBase.sol

Short successful balance or supply return data can revert cap-rule read paths

• Low Risk

The typed calls in `BalanceCapManager._balanceOf` and `TokenSupplyReader._currentSupply` do not safely translate successful but short ABI returndata into an unavailable-data result. If an initially valid configured token proxy later returns zero or insufficient bytes for `balanceOf` or `totalSupply`, ABI decoding reverts outside the `try/catch`. RuleMaxBalance recipient checks and RuleMaxTotalSupply/PoR mint checks can therefore revert instead of returning their documented unavailable-data restriction codes.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.] No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.